

Diagnostic de pannes sur pompe à chaleur

Un simulateur physique, une validation sur essais mesurés, et ce que coûte vraiment la détection

SEPTEMBRE 2026

1. Le problème, en vrai

Une pompe à chaleur qui tombe en panne d'un coup, cela arrive rarement. Ce qui arrive souvent, c'est qu'elle se dégrade sans prévenir. Le condenseur s'encrasse, une fuite lente vide une partie du fluide frigorigène, un ventilateur faiblit. La machine continue de chauffer. Elle consomme simplement de plus en plus pour le même service.

Le **COP** — coefficient de performance, le rapport entre la chaleur fournie et l'électricité consommée — glisse de 4,0 à 3,2 en quelques mois. Personne ne le voit. La facture, elle, monte de 25 %.

Le problème du technicien n'est pas de savoir *qu'il y a* un problème. C'est de savoir **lequel**. Or plusieurs pannes très différentes produisent des symptômes proches : un condenseur encrassé et un ventilateur de condenseur défaillant font tous les deux monter la pression haute et chuter le COP. L'un se règle avec un nettoyage, l'autre avec une pièce.

Ce projet répond à cette question-là : à partir des grandeurs qu'un capteur peut lire, **nommer la panne**.

2. Diagnostic n'est pas prédiction

Deux problèmes sont régulièrement confondus.

	Diagnostic (FDD)	Prédiction (PdM)
Question	Qu'est-ce qui ne va pas maintenant ?	Quand cela va-t-il casser ?
Entrée	Un point de fonctionnement	Un historique daté
Sortie	Une classe de panne	Un délai, une probabilité de survie
Donnée nécessaire	Des essais avec défaut étiqueté	Des séries temporelles jusqu'à la défaillance

Ce projet fait du diagnostic. Le choix n'est pas idéologique, il est dicté par les données disponibles : les essais mesurés utilisés ici sont des points **stationnaires**, mesurés en chambre climatique avec un défaut délibérément imposé et maintenu. Il n'y a pas d'horloge, pas de dégradation progressive, pas de moment de casse.

Fabriquer un modèle de prédiction sur ces données reviendrait à inventer un axe du temps qui n'existe pas. Le projet s'y refuse explicitement. La prédiction est une piste future, qui suppose un **autre jeu de données** (section 14).

Pour le jury. Le diagnostic et la prédiction ne se distinguent pas par le modèle mais par la donnée. Sans série temporelle allant jusqu'à la panne, un modèle de prédiction n'est pas prudent : il est faux.

3. Vue d'ensemble du système

Le trajet d'une mesure jusqu'à un diagnostic affiché :

```
simulateur      features      générateur      classifieur      moteur      API
R-410A      ->  24 grandeurs -> 5000 exemples -> GradientBoosting -> FDDEngine -> FastAPI
```

Le code est organisé en trois couches, avec une règle d'import à sens unique :

```
studies/  ->  fdd/  ->  physics/
```

- **physics/** — le modèle thermodynamique. N'importe rien du projet.
- **fdd/** — la méthode : contrat de grandeurs, entraînement, moteur de diagnostic. Partagée par toutes les études.
- **studies/** — une étude = un jeu de données, une taxonomie de pannes, un emplacement d'artefacts. Seule couche autorisée à importer les deux autres.

Cinq tests automatiques vérifient que cette règle n'est jamais violée. Les chemins de fichiers d'une étude sont centralisés dans un unique module `paths.py` par étude : aucun chemin n'est écrit en dur ailleurs.

4. Le simulateur

Faute de données de panne étiquetées en quantité, le projet **fabrique** ses exemples d'entraînement à partir des équations du cycle à compression de vapeur.

Le fluide est le **R-410A**. Ses propriétés thermodynamiques viennent de la bibliothèque CoolProp, avec des corrélations de repli si elle est absente. Un cycle est calculé à partir de trois conditions :

Condition	Plage	Sens physique
<code>T_source</code>	-10 à 20 °C	Température de la source froide (air extérieur)
<code>T_sink</code>	30 à 55 °C	Température du puits chaud (eau de chauffage)
<code>speed_ratio</code>	0,3 à 1,0	Régime du compresseur

Le point nominal du projet est **A7/W40** : air extérieur à 7 °C, eau à 40 °C.

Un défaut n'est pas un bruit ajouté après coup sur les résultats. Il est injecté **dans la physique**, en amont, et toutes les grandeurs en découlent de façon cohérente :

Défaut injecté	Paramètre dégradé
Encrassement	Coefficient d'échange UA de l'échangeur concerné
Ventilateur	Débit d'air, avec un exposant différent de l'encrassement
Sous-charge	Densité et pression d'aspiration du fluide

C'est cette différence d'exposant qui permet de distinguer un encrassement d'une panne de ventilateur : les deux dégradent l'échange, mais par des chemins différents.

Un bruit de mesure gaussien réaliste est ajouté en dernier, sur les grandeurs mesurables.

Dette connue du simulateur

Quatre défauts de modélisation ont été identifiés en confrontant les signatures simulées aux essais mesurés. Ils sont documentés, non corrigés à ce jour, et planifiés.

Défaut	Conséquence
Surcharge de fluide non modélisée	Le type de panne existe dans le code mais ne produit aucun effet
Sous-refroidissement inversé sur l'encrassement du condenseur	Une grandeur du modèle varie dans le mauvais sens
Surchauffe et température de refoulement inversées sur le défaut de ventilateur d'évaporateur	Deux grandeurs dans le mauvais sens
Limite de température de refoulement déclarée mais jamais appliquée	Des cycles à 319 °C, physiquement impossibles, dans le domaine d'entraînement

Ces défauts sont énoncés ici parce qu'ils ont été **mesurés**, et parce qu'un modèle dont on connaît les limites vaut mieux qu'un modèle dont on les ignore.

5. Les 24 grandeurs

Le classifieur ne voit pas un cycle. Il voit un vecteur de 24 nombres, défini une fois pour toutes dans le contrat `FEATURE_COLUMNS`.

#	Nom	Unité	Origine	Rôle
1	T_ambient	°C	Condition	Identique à T_source
2	T_setpoint	°C	Condition	Identique à T_sink
3	compressor_speed_ratio	-	Condition	Régime demandé
4	P_evap	bar	Mesure	Pression d'évaporation
5	P_cond	bar	Mesure	Pression de condensation
6	T_evap	°C	Mesure	Température de saturation, côté froid
7	T_cond	°C	Mesure	Température de saturation, côté chaud
8	T_suction	°C	Mesure	Entrée compresseur
9	T_discharge	°C	Mesure	Sortie compresseur
10	superheat	K	Mesure	Surchauffe
11	subcooling	K	Mesure	Sous-refroidissement
12	compression_ratio	-	Dérivé	P_{cond} / P_{evap}
13	W_comp	W	Mesure	Puissance absorbée
14	Q_cond	W	Mesure	Puissance restituée
15	COP	-	Dérivé	Q_{cond} / W_{comp}
16	delta_T_evap	K	Dérivé	$T_{source} - T_{evap}$
17	delta_T_cond	K	Dérivé	$T_{cond} - T_{sink}$
18	pressure_ratio	-	Dérivé	P_{cond} / P_{evap}
19	capacity_ratio	-	Dérivé	Q_{cond} rapporté au nominal
20	d_T_discharge	K	Résidu	Écart au cycle sain
21	d_superheat	K	Résidu	Écart au cycle sain
22	d_subcooling	K	Résidu	Écart au cycle sain
23	d_COP	-	Résidu	Écart au cycle sain
24	d_W_comp	W	Résidu	Écart au cycle sain

Deux définitions utiles. La *surchauffe* est le nombre de degrés au-dessus de la température d'ébullition à la sortie de l'évaporateur : elle dit si le fluide est bien entièrement vaporisé. Le *sous-refroidissement* est le symétrique à la sortie du condenseur : il dit combien de liquide s'y accumule.

Une redondance. Les grandeurs 12 et 18 sont le même nombre, vérifié à la précision machine sur tous les points de fonctionnement testés. Le modèle dispose donc de 23 entrées indépendantes, pas 24. La correction est planifiée.

Les cinq résidus

Un **résidu** est une soustraction :

`résidu = valeur observée - valeur qu'aurait une machine saine, dans les mêmes conditions`

L'analogie est celle de la fièvre. Dire « 37,8 °C » ne veut rien dire sans savoir de qui l'on parle. Dire « 0,9 °C au-dessus de sa propre température habituelle » est un signal, quel que soit l'individu.

C'est exactement le raisonnement. Une surchauffe de 12 K ne dit rien : elle dépend de la machine, de la saison, de la charge. Une surchauffe **1,7 K au-dessus de ce que cette machine-ci ferait en bonne santé, à cet instant-ci** est une signature de sous-charge.

Cette approche porte un nom dans la littérature du domaine : la méthode des résidus de Li et Braun. Elle est au cœur du projet, et la section 11 montre qu'elle est ce qui fonctionne — avec une condition qui n'avait pas été anticipée.

6. Les pannes traitées

L'étude synthétique produit six classes :

Classe	Description
<code>Normal</code>	Fonctionnement sain
<code>Condenser_Fouling</code>	Condenseur encrassé
<code>Evaporator_Fouling</code>	Évaporateur encrassé
<code>Refrigerant_Undercharge</code>	Fuite, charge insuffisante
<code>Condenser_Fan_Fault</code>	Débit d'air condenseur réduit
<code>Evaporator_Fan_Fault</code>	Débit d'air évaporateur réduit

Deux types de panne supplémentaires existent dans le code — surcharge de fluide et fuite de clapet de compresseur — avec une branche d'injection complète, mais ne sont **jamais générés**. Ce sont des classes fantômes. Leur sort est une décision assumée du projet, pas un oubli.

Les essais mesurés utilisés en validation couvrent deux pannes que le simulateur ne produit pas : la **surcharge** et la **restriction de ligne liquide**.

7. Le modèle

Algorithme	Gradient Boosting, calibré en probabilité
Alternative testée	Random Forest
Entraînement	5000 cycles simulés, 30 % en test
Sélection	Recherche sur grille, validation croisée stratifiée

Le choix ne repose pas sur un empilement d'algorithmes. Sur des données simulées où les classes sont déjà nettement séparées, un modèle supplémentaire n'apporte rien de mesurable : Random Forest obtient 99,7 % contre 99,8 %. L'écart n'est pas significatif. Multiplier les modèles aurait donné une illusion de rigueur ; le travail utile était ailleurs, dans la validation.

Une contrainte technique importante. Le modèle entraîné est versionné dans le dépôt sous forme de fichier sérialisé. Ce format dépend de la version exacte de la bibliothèque d'apprentissage. La version est donc **épinglée à `scikit-learn==1.6.1`** dans les dépendances. Installer une autre version rend le modèle illisible, avec un message d'erreur qui ne ressemble en rien à un problème de version. Toute mise à jour impose un réentraînement.

8. L'interface

L'API expose deux familles de routes, de nature différente :

Famille	Routes	Rôle
Inférence	<code>/health</code> , <code>/predict</code> , <code>/simulate</code> , <code>/live</code>	Le diagnostic proprement dit
Tableau de bord	17 routes <code>/api/*</code>	Alimenter les vues du front

Le contrat d'entrée de `/predict` est strict : il énumère les 24 grandeurs et **refuse tout champ inconnu**. C'est un choix défensif — une grandeur mal nommée est rejetée au lieu d'être silencieusement ignorée — mais cela signifie que modifier la liste des grandeurs est une **rupture de compatibilité**, à traiter comme telle.

Le front, lui, est générique : il lit la liste des grandeurs depuis l'API plutôt que de la coder en dur. Il suivra une évolution du contrat sans modification.

9. Les données mesurées

La validation s'appuie sur une campagne d'essais publique du NIST, menée sur des pompes à chaleur résidentielles en chambre climatique, avec défauts imposés.

Essais	7375
Grandeurs mesurées	98 colonnes, côté air et côté fluide
Machines	Deux, notées 14 et 16 SEER
Retenus	5386 essais à défaut unique, sur 6 classes
Mode	Refroidissement

Le **SEER** est un indice d'efficacité saisonnière : deux valeurs signifient simplement deux machines de performances différentes. Cette diversité est un atout, pas une nuisance : elle permet de tester si un modèle appris sur une machine fonctionne sur l'autre.

Deux pièges de lecture

Les étiquettes de panne sont dans les données. Cinq colonnes indiquent le niveau de défaut imposé. Elles constituent la réponse à trouver : les inclure parmi les grandeurs d'entrée reviendrait à donner le corrigé avec l'énoncé. Elles sont explicitement exclues, et un test le vérifie.

Un capteur n'est pas instrumenté sur une des deux machines. La colonne nommée « pression au port d'aspiration » contient en réalité la pression de refoulement sur la machine 16 SEER, soit 55 % des essais. Utilisée sans vérification, elle produit un rapport de pression égal à 1,00 — physiquement impossible — sans lever la moindre erreur. Une autre colonne, cohérente sur les deux machines, est utilisée à la place.

Les classeurs de données ne sont pas versionnés dans le dépôt en raison de leur taille. Les adresses de téléchargement et la table de correspondance complète figurent dans la documentation technique.

10. Comment on mesure une performance

Un score n'a de sens qu'accompagné de son protocole.

Validation croisée aléatoire. On mélange tous les essais, on en cache une partie, on entraîne sur le reste. Simple, standard, et trompeur ici : les essais des deux machines se retrouvent des deux côtés. Le modèle peut apprendre à reconnaître *la machine* plutôt que *la panne*.

Validation par machine (leave-one-machine-out). On entraîne sur une machine, on teste sur l'autre, puis on inverse. Le modèle ne peut plus s'appuyer sur l'identité de l'installation.

L'analogie est celle d'un élève. Réviser sur les annales puis composer sur un sujet tiré des mêmes annales donne une bonne note. Composer sur un sujet d'un autre établissement mesure ce qu'il a réellement compris.

Résultats mesurés, quatre jeux de grandeurs, même algorithme, mêmes données :

Jeu de grandeurs	CV aléatoire	Par machine
Grandeurs brutes	0,954	0,333
Brutes et résidus	0,973	0,562
Résidus seuls	0,937	0,602
Résidus et conditions	0,945	0,594

Référence basse, en prédisant toujours la classe la plus fréquente : **0,251**.

Deux lectures. Le jeu qui obtient **le meilleur score en validation aléatoire n'est pas celui qui se transfère le mieux** : classer des modèles sur un tirage aléatoire aurait conduit à retenir la mauvaise conception. Et ajouter les grandeurs absolues aux résidus **dégrade** le transfert, parce que les valeurs absolues permettent au modèle de ré-identifier la machine.

Pour le jury. Un écart de 0,95 à 0,60 entre deux protocoles, sur le même modèle et les mêmes données, ne mesure pas le modèle. Il mesure le protocole. Tout score annoncé dans ce projet est accompagné du sien.

11. Le résultat qui change le discours

Les résidus font passer la détection de 0,333 à 0,602. C'est la validation empirique, sur des mesures réelles, du choix de conception du projet.

Mais une vérification ultérieure a montré que ce 0,602 reposait sur une hypothèse implicite : la référence saine — la « température habituelle » de l'analogie — était calculée à partir d'essais sains **de la machine testée**.

En rejouant l'expérience avec une référence saine issue uniquement de la machine d'entraînement, c'est-à-dire en simulant une machine réellement inconnue :

Origine de la référence saine	Score	F1 macro
Essais sains de la machine cible	0,602	0,479
Référence transférée depuis l'autre machine	0,318	0,290
Polynôme d'ordre 2 avec point de rosée	0,302	0,254
Forêt aléatoire avec point de rosée	0,265	0,248

Plusieurs formes de modèle de référence ont été essayées, y compris celle décrite par le NIST lui-même. **Aucune ne lève le plafond**. Le résultat n'est pas un échec : c'est une mesure.

La conclusion réoriente le projet :

Le diagnostic par résidus fonctionne, à condition de disposer d'une calibration saine sur la machine en service. Il ne se transfère pas d'une pompe à chaleur inconnue vers une autre.

C'est une contrainte de déploiement industriel, pas une limite de l'algorithme. Elle a une conséquence pratique directe : avant de diagnostiquer une machine, il faut l'observer en bonne santé. La question devient : **combien de temps ?**

Pour le jury. L'écart entre 0,602 et 0,318 est le résultat principal de ce travail. Il ne dit pas que la méthode échoue : il dit ce qu'elle exige pour fonctionner. Une méthode dont on connaît le prix est utilisable ; une méthode dont on ignore la condition ne l'est pas.

12. Budget de calibration — mesuré

Combien d'essais sains de la machine cible faut-il pour passer de 0,318 à 0,602 ?

Leave-one-machine-out, les deux sens, vingt tirages. Les n sains qui calibrent la référence sont **retirés du test**. Bornes retrouvées : $n = 0 \rightarrow 0,318$; $n = \text{all} \rightarrow 0,602$.

n essais sains (machine cible)	Accuracy	F1 macro
0 (référence transférée)	0,318	0,290
5	0,314	0,289
10	0,377	0,324
20	0,409	0,349
50	0,456	0,380
tous (~625–727)	0,602	0,479

Grandeur	Valeur
Essais sains nécessaires pour récupérer 90 % du gain	tous les sains de la machine

À $n = 50$ on n'a récupéré que **~49 %** de l'écart. Le seuil 0,574 n'est atteint qu'avec l'ensemble des essais sains. Une poignée de mesures de mise en service ne calibre pas : il faut couvrir le domaine de fonctionnement. Répartir les n points sur les températures aide surtout à petit n ($n = 5$: 0,391 au lieu de 0,314).

Figure : docs/calibration_budget.png . Notebook : EDA/EDA_NIST_calibration.ipynb .

13. Ce que ce projet ne fait pas

Par honnêteté, et parce que chacune de ces limites a été vérifiée plutôt que supposée :

- **Il ne détecte pas les pannes à 99 % sur le terrain.** Le 99,8 % mesure la séparabilité des signatures à l'intérieur du modèle physique, sur données simulées.
- **Il ne prédit pas les pannes futures.** Les essais mesurés sont stationnaires, sans axe du temps.
- **Il ne fonctionne pas sur une machine inconnue sans calibration.** C'est le résultat de la section 11.
- **Il ne compare pas ses grandeurs absolues aux essais mesurés.** Le simulateur travaille en mode chauffage sur une machine air-eau, les essais en mode refroidissement sur une machine air-air. Le recouvrement des domaines de fonctionnement est de **5,3 %**. Seules les tendances sont comparables — et sur ce terrain, l'accord des sens de variation entre simulation et mesure est de **12 sur 16**.

14. Perspective

Une troisième étude est envisagée, consacrée à la **prédiction de dégradation**. Elle est conditionnée à l'obtention d'un jeu de données adapté : des séries temporelles datées, allant jusqu'à une intervention ou une défaillance.

Elle ne sera pas construite sur les essais actuels. Ceux-ci n'ont pas d'axe du temps, et leur en inventer un produirait exactement le type de résultat que ce projet s'attache à ne pas produire.

L'architecture en études indépendantes est prévue pour cela : une nouvelle étude s'ajoute sans toucher aux précédentes, avec ses données, sa taxonomie et ses métriques propres.

15. Reproduire

```
pip install -r requirements.txt
uvicorn api.app:app --reload
cd web && npm install && npm run dev
```

La suite de tests automatiques compte 42 tests et doit rester intégralement verte. Elle couvre les invariants thermodynamiques, le contrat de grandeurs, les règles d'architecture et les contrats de l'API.

Les notebooks d'exploration des données mesurées sont dans le dépôt. Les classeurs de données, eux, doivent être téléchargés depuis la source publique indiquée dans la documentation technique.

Les chiffres, en un tableau

Mesure	Valeur	Ce qu'elle signifie
Simulé, validation aléatoire	99,8 %	Séparabilité dans le modèle physique
Mesuré, validation aléatoire	0,95	Surestime : mélange les deux machines
Mesuré, par machine, référence calibrée	0,602	Avec essais sains de la machine cible
Mesuré, par machine, référence transférée	0,318	Sur une machine réellement inconnue
Classe majoritaire	0,251	Référence basse
Accord des sens de variation	12 / 16	Simulation contre mesure
Recouvrement des domaines	5,3 %	Interdit la comparaison des valeurs absolues